2026

C Workshop

Functions , pointers and structures:



Welcome to SESSION 4

Wameedh SC

Agenda

introduction
What are functions , pointers and structures, and why do they matter ?

Functions

1 -Function declaration, definition, and call
- variables scope (global vars vs local vars)
2 -Passing parameters (by value and by reference)
3 -Returning values from functions
4 -void functions vs datatype forms
5 -Introduction to structures
6 -Introduction to pointers

```c
#include <stdio.h>

int main()
{

    printf("Welcome to wameedh scientific club \n");
    printf("This is the last session of c programming workshop\n");
    printf("Wish me luck :)\n");

    return 0;
}
```

introduction.

1- Functions :

A function is a reusable block of code that performs a specific task.
Imagine it like a math function : you give it input, it does work, and it gives output.

```c
#include <stdio.h>

void sayHello() {
    printf("Hello!\n");
}

int main() {
    sayHello();  // calling the function
    return 0;
}
```

Why do they matter ?

- Organize your code (split big problems into small pieces)
- Avoid repetition (write once, use many times)
- Make programs easier to read and debug

another example

```c
#include <stdio.h>

void add() {          // Function definition
    int a = 5, b = 7;
    printf("Sum = %d\n", a + b);
}

int main() {
    add();            // Function call
    add();            // We can call it again!
    return 0;
}
```

before moving to the next phase

lets understand variables scope

what is a global variable and what is a local variable

lets run this code and discover what happen

it will give us an error

why ?

```c
#include <stdio.h>

void function1()
{
    int local = 1;
}

void main()
{
   function1();
   local = 2;
}
```

lets understand the hierarchy of the memory

Stack	all the functions and local variables are stored here (Grows downwards ↓)
Free	(Unused) it will be croped when stack or heap extends
Heap	(Grows upwards ↑) we use pointers here
Global Vars	(Data Segment)
Code	(Text Segment)

```c
#include <stdio.h>

int x = 10;  // global variable

void show() {
    int x = 5;  // local variable
    printf("Inside function: x = %d\n", x);
}

int main() {
    show();
    printf("In main: x = %d\n", x);
}
```

```c
#include <stdio.h>

int x = 10;  // global variable

void show() {
    x = 5;    // this will change the global variable
}

int main() {
    show();
    printf("x = %d\n", x);  // will print 5
}
```

Local Variable Dies After Function Ends

```c
#include <stdio.h>

void test() {
    int a = 1;   // created each call
    a++;
    printf("a = %d\n", a);
}

int main() {
    test();
    test();
    test();
}
```

2 Returning values from functions :

Returning a value means the function sends a result back to the part of the program that called it.

```c
int add(int a, int b) {
    return a + b;
}

int main() {
    int result = add(3, 4);
    return 0;
}
// function returns 7
```

```c
int square(int n) {
    return n * n;
}

int main() {
    int result = square(9);
    return 0;
}
// function returns 81
```

but wait

you can see that we used a datatype in function

Difference Between void Functions and Datatype Functions

Feature :	Datatype functions :	void functions:
Returns a value?	✓ Yes	✗ No
Has a return type?	int, float, char, double...	Always void
Uses return?	✓ Yes, to send back a value	Optional, only to exit function early
Can be used in expressions?	✓ Yes (`x = add(a,b)`)	✗ No
Best used for...	Calculations, data processing, logic	Displaying, printing, modifying data

More examples

datatype function (int)

we used it because we want it to return a numerical value

```c
int strLength(char str[]) {
    int count = 0;

    while (str[count] != '\0') {   // loop until end of string
        count++;
    }

    return count;  // return the length
}
```

```c
int main() {
    char Club_Name[] = "Wameedh Scientific Club";
    int len1 = strLength(Club_Name);

    char Worshop_name[] = "C Programming Workshop";
    int len2 = strLength(Worshop_name);
    printf("Length of club name = %d \n", len1);
    printf("Length of Workshop name = %d", len2);

    return 0;
}
```

More examples

void function

we want the function to do some work (like printing, changing variables, drawing something, etc.) but we don’t need a value back from it.

```c
void strCopy(char dest[], char src[]) {
    int i = 0;

    while (src[i] != '\0') {
        dest[i] = src[i];     // copy each char
        i++;
    }

    dest[i] = '\0';           // add null terminator
}
```

```c
int main() {
    char source[] = "Hello";
    char target[10];

    strCopy(target, source);

    printf("%s", target);
    return 0;
}
```

Passing by value and by reference

```c
#include <stdio.h>

void increment(int x) {
    x = x+ 1;
}

void main()
{
    int a = 2;
    printf("a before the incrementing function : %d \n\n" , a);
    increment(a);
    printf("a after the incrementing function : %d \n\n" , a);

}
```

expected output is : 3

but no...

Passing by value and by reference

```c
#include <stdio.h>

void printNumber(int num) {
    num = 100;
    printf("Number inside function: %d\n", num);
}

int main() {
    int number = 50;
    printNumber(number);
    printf("Number in main: %d\n", number);
    return 0;
}
```

expected output is :
100
and again nooo :)

swap function example :

in main fct

```c
#include <stdio.h>
int main() {
    int x = 10, y = 20;
    int temp;
    printf("Before swap: x = %d, y = %d\n", x, y);
    temp = x;
    x = y;
    y = temp;
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

```
Before swap: x = 10, y = 20
After swap: x = 20, y = 10
```

in independant fct

```c
#include <stdio.h>
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}
int main() {
    int x = 10, y = 20;
    swap(x, y);  // try to swap
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

```
x = 10, y = 20
```

The solution is :

```c
#include <stdio.h>

void swap(int *a, int *b) {  // pass by reference
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    swap(&x, &y);  // swap using addresses
    printf("x = %d, y = %d\n", x, y);  // values swapped
    return 0;
}
```

```c
#include <stdio.h>

void changeNumber(int *num) {
    *num = 100; // changes the original variable
    printf("Inside function: %d\n", *num);
}

int main() {
    int number = 50;
    changeNumber(&number); // pass by reference
    printf("In main: %d\n", number); // original changed
    return 0;
}
```

```
Inside function: 100
In main: 100
```

Why swap doesn’t work without * ?

Without * (Pass by Value) :

The function gets copies of the variables.
Any changes inside the function do not affect the original variables.
Analogy: Giving a friend a copy of your homework — changes don’t affect the original.

With * (Pass by Reference) :

The function gets the addresses of the variables.
Changes inside the function modify the original values.
Analogy: Giving a friend the actual homework — any changes affect the original.

Lets move on to pointers

The pointer is a variable that stores a memory address.

Not a value — an address of where the value is stored.

Think of it like:

- A normal variable = the house
- A pointer = the address of the house

```c
#include <stdio.h>

int main() {
    int x = 10;
    int *p = &x;   // p stores the address of x

    printf("x = %d\n", x);
    printf("Address of x = %p\n", &x);
    printf("p = %p\n", p);     // same as &x
    printf("*p = %d\n", *p);   // value stored at that address

    return 0;
}
```

Array traversal using pointers :

```c
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;  // points to arr[0]

    printf("%d\n", *p);      // 10
    printf("%d\n", *(p+1));  // 20
    printf("%d\n", *(p+2));  // 30

    return 0;
}
```

Thank you for your attention

Powered by Wameedh SC



Yanis Boubekeur